

파일 다루기

학습 체크리스트 (1/2)

☐ `multipart/form-data` 전송과 `MultipartFile` 변환 흐름 이해

☐ 멀티파트 용량 제한 설정과 기본값 파악

☐ 저장 책임을 `FileStore`로 분리하는 이유와 구조 습득

☐ UUID 파일명과 경로 이탈 방어의 필요성 이해

학습 체크리스트 (2/2)



@RequestParam · 폼 객체로 파일을 받는 방법 습득



타임리프 업로드 폼의 필수 속성과 제약 파악



DB에 저장 파일명만 기록하고 URL을 조립하는 원칙 이해



1:1 프로필 이미지와 1:N 첨부 이미지의 설계 차이 습득

파일 업로드의 원리

일반 입력값과 파일의 차이

- 문자열 vs 바이너리: 텍스트 입력값은 문자열이지만 파일은 바이트로 이루어진 바이너리 데이터
- 전송 방식 분리: 브라우저가 폼을 `multipart/form-data` 로 나누어 파트별로 전송
- 컨트롤러 전달: Spring MVC가 파일 파트를 `MultipartFile` 객체로 변환해 컨트롤러에 전달

업로드 요청부터 화면 표시까지의 흐름



멀티파트 폼 데이터 개념 정리

- 일반 폼과의 차이: 텍스트 필드와 파일 파트가 하나의 요청 안에 함께 담겨 전송됨
- 경계 문자열: 각 파트를 구분하기 위해 요청 헤더에 boundary 값이 포함됨

멀티파트 폼 데이터 (multipart/form-data)

텍스트와 파일을 여러 파트로 나누어 하나의 HTTP 요청으로 함께 전송하는 인코딩 방식

파일은 저장소에, DB에는 이름만

- 이번 차시 저장 위치: 실습 편의를 위해 `src/main/resources/static/upload`에 직접 저장
- DB 저장 값: 원본 파일명과 저장 파일명만 기록하고 파일 본문은 파일 시스템에 저장
- 실서비스 확장: 운영에서는 애플리케이션 외부 경로나 객체 스토리지를 사용

별도 라이브러리가 필요 없는 이유

- 자동 구성: 웹 스타터가 있으면 멀티파트 처리가 별도 설정 없이 자동 구성됨
- 기본 용량 제한: 파일 1개 최대 1MB, 요청 전체 최대 10MB
- 다중 파일 주의: 여러 파일을 한 번에 받으면 요청 전체 크기를 더 크게 잡아야 함

업로드 용량 상한 설정

```
1  spring:
2    servlet:
3      multipart:
4        max-file-size: 5MB
5        max-request-size: 20MB
```

MultipartFile 주요 메서드

메서드	의미
<code>isEmpty()</code>	파일이 첨부되지 않았는지 확인
<code>getOriginalFilename()</code>	클라이언트가 보낸 원본 파일명 조회
<code>getContentType()</code>	클라이언트가 보낸 MIME 타입 조회
<code>getSize()</code>	파일의 바이트 크기 조회
<code>transferTo(Path)</code>	지정 경로로 파일 내용을 저장

클라이언트가 보낸 값은 믿지 않는다

- 신뢰 불가 값: `getOriginalFilename()`, `getContentType()` 은 모두 클라이언트가 보낸 값
- 원본 이름 그대로 저장 시 위험: 이름 충돌과 경로 조작(path traversal) 문제 발생 가능
- 서버 측 대응: 서버가 UUID로 새 파일명을 생성해 저장

저장 책임 분리하기

컨트롤러가 파일 저장을 직접 하지 않는 이유

- **중복 발생:** 파일명 생성, 저장 경로 계산, 확장자 검증이 컨트롤러마다 반복됨
- **변경에 취약:** 저장 방식(로컬 → 클라우드 등)이 바뀌면 관련된 모든 컨트롤러를 수정해야 함
- **책임 분리:** 파일 저장은 `FileStore` 한 곳에서만 담당하고, 컨트롤러는 요청 처리에 집중

원본 이름과 저장 이름을 함께 다루는 이유

- 사용자용 이름: 화면에 보여주거나 다운로드할 때는 원본 파일명이 필요함
- 서버용 이름: 실제 파일에 접근할 때는 충돌 없는 저장용 이름이 필요함
- 작은 객체로 묶기: 두 이름을 함께 반환할 작은 객체가 있으면 이후 처리가 단순해짐

원본 이름과 저장 이름을 담는 UploadFile

```
1 public record UploadFile(  
2     String originalName,  
3     String storedName  
4 ) {  
5     public String url() {  
6         return "/upload/" + storedName;  
7     }  
8 }
```

FileStore가 파일을 저장하는 4단계

단계	목적
형식 검증	확장자·크기·파일 시그니처를 함께 확인함
디렉터리 준비	저장 폴더가 없으면 새로 생성함
UUID 이름 생성	파일명 충돌과 경로 추측을 방지함
실제 파일 저장	검증을 통과한 파일만 지정 경로에 씀

FileStore 컴포넌트의 기본 골격

```
1  @Component
2  public class FileStore {
3
4      private static final Set<String> ALLOWED_EXTENSIONS =
5          Set.of("jpg", "jpeg", "png", "gif", "webp");
6
7      private final Path uploadPath =
8          Path.of("src/main/resources/static/upload")
9              .toAbsolutePath().normalize();
10 }
```

허용 확장자 목록으로 1차 검증

```
1  if (file == null || file.isEmpty()) return null;
2  String originalName = Objects.requireNonNull(file.getOriginalFilename());
3  String safeName = StringUtils.getFilename(
4      StringUtils.cleanPath(originalName));
5  String extension = StringUtils.getFilenameExtension(safeName);
6  if (extension == null) throw new IllegalArgumentException("확장자 없음");
7  extension = extension.toLowerCase(Locale.ROOT);
8  if (!ALLOWED_EXTENSIONS.contains(extension))
9      throw new IllegalArgumentException("지원하지 않는 형식입니다.");
```

UUID 이름 생성과 경로 이탈 검사 후 저장

```
1  Files.createDirectories(uploadPath);
2  String storedName = UUID.randomUUID() + "." + extension;
3  Path target = uploadPath.resolve(storedName).normalize();
4
5  if (!target.startsWith(uploadPath)) {
6      throw new IllegalArgumentException("잘못된 저장 경로입니다.");
7  }
8
9  file.transferTo(target);
10 return new UploadFile(safeName, storedName);
```

저장 과정에 필요한 방어 계층

- `createDirectories`: 폴더가 없으면 만들고, 이미 있으면 그대로 통과함
- UUID 이름: 사용자 입력을 저장 경로에 쓰지 않고 이름 충돌도 방지함
- 경로 검사: 정규화한 대상이 `static/upload` 폴더 안에 있는지 확인함
- 콘텐츠 검증: 확장자·크기와 함께 파일 시그니처를 검사하고 필요하면 재인코딩함

컨트롤러와 업로드 폼

파일을 받는 세 가지 방법

방법	사용 상황
<code>@RequestParam MultipartFile</code>	파일 하나만 단독으로 받을 때
폼 객체의 <code>MultipartFile</code> 필드	제목 등 다른 입력값과 파일을 함께 받을 때
<code>List<MultipartFile></code>	여러 파일을 한 번에 받을 때

업로드 폼을 여는 컨트롤러 뼈대

```
1  @Controller
2  @RequestMapping("/files")
3  class FileController {
4      private final FileStore fileStore;
5      FileController(FileStore fileStore) {
6          this.fileStore = fileStore;
7      }
8      @GetMapping("/new")
9      String form() { return "files/form"; }
10 }
```

업로드 요청 처리와 결과 리다이렉트

```
1  @PostMapping
2  String upload(@RequestParam("image") MultipartFile image,
3               RedirectAttributes ra) throws IOException {
4      if (!image.isEmpty()) {
5          UploadFile uploadFile = fileStore.store(image);
6          ra.addFlashAttribute("uploadFile", uploadFile);
7      }
8      return "redirect:/files/new";
9  }
```

업로드 후 리다이렉트하는 이유

- PRG 패턴: 새로고침 시 같은 업로드 요청이 재전송되는 것을 방지
- 일회성 메시지: 결과 메시지는 `RedirectAttributes`의 flash 속성으로 한 번만 전달
- 화면에 넘기는 값: 저장된 파일명으로 만든 URL을 화면 렌더링에 전달

제목과 이미지를 함께 받는 폼 객체

```
1  class BookForm {  
2      private String title;  
3      private List<MultipartFile> images = new ArrayList<>();  
4  
5      // getter.setter  
6  }
```

폼 객체로 여러 파일 한 번에 받기

```
1  @PostMapping("/books")
2  String create(@ModelAttribute BookForm form) {
3      bookService.create(form.getTitle(), form.getImages());
4      return "redirect:/books";
5  }
```

파일 업로드 폼이 지켜야 할 조건

- 필수 속성: `<form>` 에 `method="post"` 와 `enctype="multipart/form-data"` 반드시 지정
- 이름 일치: `input` 의 `name` 이 컨트롤러 파라미터 이름과 정확히 일치해야 함
- `accept` 속성의 한계: `accept="image/*"` 는 선택 편의 기능일 뿐, 서버 측 검증을 대신하지 않음

파일 하나를 올리는 타임리프 폼

```
1 <form th:action="@{/files}" method="post" enctype="multipart/form-data">
2     <input type="file" name="image" accept="image/*" required>
3     <button type="submit">업로드</button>
4 </form>
```

업로드 결과 메시지와 이미지 출력

```
1 <p th:if="${uploadFile}"
2   th:text="${uploadFile.originalName()} + ' 업로드 완료'"></p>
3
4 
```

검증 실패로 폼을 다시 보여줄 때

- 선택 파일 소실: 브라우저 보안 정책상 선택했던 파일은 유지되지 않아 사용자가 다시 선택해야 함
- 텍스트 값 유지: 제목 등 텍스트 입력값은 폼 객체를 다시 넘겨 유지
- 다중 선택 지원: 여러 파일을 받으려면 `input` 에 `multiple` 속성 추가

여러 파일을 선택하는 타임리프 폼

```
1 <form th:object="${bookForm}" th:action="@{/books}"
2     method="post" enctype="multipart/form-data">
3     <input type="text" th:field="*{title}">
4     <input type="file" th:field="*{images}" multiple accept="image/*">
5     <button type="submit">등록</button>
6 </form>
```

DB에는 무엇을 저장하는가

DB에 저장하는 두 가지 값

항목	예시	용도
원본 파일명	<code>내 사진.png</code>	사용자에게 보여줄 이름
저장 파일명	<code>550e8400- ... png</code>	실제 파일과 URL을 찾는 이름

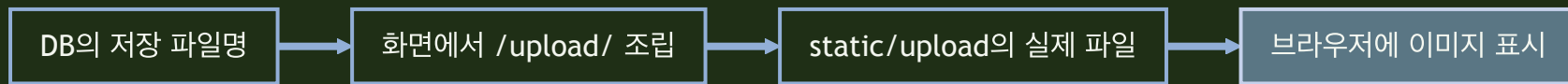
전체 경로를 저장하지 않는 이유

- 환경 의존 경로 문제: `src/main/resources/static/upload/ ...` 같은 전체 경로는 환경이 바뀌면 깨짐
- 저장하는 값: 저장 파일명만 DB에 기록
- URL 조립 방식: 화면에서 앞에 `/upload/` 를 붙여 URL을 만들어 사용

저장 파일명으로 이미지 URL 만들기

```
1 
```

파일 실체, DB, 화면의 관계



1:1 관계 — 사용자와 프로필 이미지

- 단일 이미지: 사용자 한 명에 프로필 이미지가 하나면 별도 테이블 없이 User에 파일명 컬럼 두 개
- null 허용: 이미지가 없을 수 있으므로 두 컬럼 모두 null 허용
- 부분 갱신: 새 파일을 선택하지 않으면 기존 값 유지

User 엔티티의 프로필 이미지 필드

```
1  @Entity
2  class User {
3      @Id @GeneratedValue
4      private Long id;
5      private String email;
6      private String profileImageOriginalName;
7      private String profileImageStoredName;
8      // getter 생략
9  }
```

프로필 이미지 정보를 바꾸는 메서드

```
1  class User {  
2      void changeProfileImage(UploadFile uploadFile) {  
3          this.profileImageOriginalName = uploadFile.originalName();  
4          this.profileImageStoredName = uploadFile.storedName();  
5      }  
6  }
```

프로필 이미지 변경 서비스

```
1  @Service
2  @Transactional
3  class UserService {
4      void changeProfileImage(Long id, MultipartFile file) throws IOException {
5          if (file.isEmpty()) return;
6          User user = userRepository.findById(id).orElseThrow();
7          UploadFile uploadFile = fileStore.store(file);
8          user.changeProfileImage(uploadFile);
9      }
10 }
```

1:N 관계 — 도서와 여러 이미지

- 다중 이미지: 도서 하나에 이미지가 여러 개면 BookImage 엔티티를 따로 둬
- 연관관계의 주인: 외래 키를 가진 BookImage가 연관관계의 주인
- 순서 정보: 정렬 순서를 BookImage에 함께 저장

Book 엔티티의 이미지 컬렉션

```
1  @Entity
2  class Book {
3      @Id @GeneratedValue
4      private Long id;
5      private String title;
6      @OneToMany(mappedBy = "book", cascade = CascadeType.ALL,
7                  orphanRemoval = true)
8      private List<BookImage> images = new ArrayList<>();
9  }
```

양방향 연관관계를 맞추는 addImage

```
1  class Book {  
2      void addImage(BookImage image) {  
3          images.add(image);  
4          image.setBook(this);  
5      }  
6  }
```

BookImage 엔티티

```
1  @Entity
2  class BookImage {
3      @Id @GeneratedValue
4      private Long id;
5      @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "book_id")
6      private Book book;
7      private String originalName, storedName;
8      private int sortOrder;
9      // 생성자·getter 생략
10 }
```

도서와 이미지 여러 장 저장

```
1  int sortOrder = 0;
2  for (MultipartFile file : files) {
3      if (file.isEmpty()) continue;
4      UploadFile uploadFile = fileStore.store(file);
5      book.addImage(new BookImage(uploadFile, sortOrder++));
6  }
7  bookRepository.save(book);
```

cascade와 orphanRemoval이 하는 일

- **cascade**: Book을 저장하면 컬렉션에 담긴 BookImage도 함께 저장
- **orphanRemoval**: 관리 중인 컬렉션에서 빠진 BookImage는 플러시 때 DB에서 삭제됨
- **한계**: `static/upload`의 실제 파일은 지워지지 않아 파일 삭제 코드를 별도로 호출해야 함

이미지 목록 출력

```
1 <div th:each="image : ${book.images}">
2     
4 </div>
```

실습 방식과 실제 서비스의 차이

항목	실습(<code>static/upload</code>)	실무
저장 위치	소스 트리에서 바로 확인 가능한 실습용 경로	외부 볼륨·객체 스토리지로 교체
안정성	JAR 실행·재빌드·다중 인스턴스에는 부적합	저장소 키로 파일을 안정적으로 관리
접근 제어	<code>/upload/**</code> 는 URL을 아는 누구나 접근 가능	접근 제어를 별도로 구성

학습 요약 (1/2)

- 멀티파트 전송과 MultipartFile:
 - HTML form의 파일 전송 방식과 이를 받는 MultipartFile 이해
- 저장 책임의 분리:
 - 실제 파일 저장 로직을 FileStore로 분리하여 컨트롤러/서비스와 관심사 분리
- 컨트롤러와 업로드 폼의 필수 조건:
 - `enctype="multipart/form-data"` 와 컨트롤러 파라미터 매핑 조건 습득

학습 요약 (2/2)

- DB에는 파일명만 저장:
 - 원본 파일명과 저장 파일명을 기록하고 전체 경로는 저장하지 않는 원칙 이해
- 1:1과 1:N 설계 차이:
 - 단일 이미지는 컬럼으로, 다중 이미지는 별도 엔티티로 설계하는 기준 습득
- 실습과 실무 저장 방식의 차이:
 - `static/upload` 실습 방식과 객체 스토리지 기반 실무 방식의 차이 이해